```python
import os
from pickle import dump
from tabulate import tabulate

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split, cross_validate

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    confusion_matrix,
    roc_curve,
    roc_auc_score,
)
from sklearn.pipeline import Pipeline

# Some constants
DATA_PATH = os.path.join("balanced_sentiment_dataset.csv")
PLOT_DIR = "plots"
MODEL_DIR = "models"
TABLE_DIR = "tables"
CM_PLOT_FNAME = "-cm.svg"
ROC_PLOT_FNAME = "-roc.svg"
MODEL_FNAME = "-model.pkl"
TABLE_FNAME = "table.tex"


# Function to save a confusion matrix plot from a given model
def cm_plot(y_true, y_pred, mname):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(f"{mname}-cm")
    sns.heatmap(cm, annot=True)
    plt.title(f"Confusion Matrix for {mname}")
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.savefig(os.path.join(PLOT_DIR, mname + CM_PLOT_FNAME))


# Function to save an ROC plot from a given model
def roc_plot(y_true, y_score, mname):
    fpr, tpr, _ = roc_curve(y_true, y_score)
    plt.figure(f"{mname}-roc")
    plt.plot([0, 1], [0, 1], linestyle=":", color="gray")
    plt.plot(fpr, tpr)
    plt.title(f"ROC Curve for {mname}")
    plt.xlabel("False positive rate")
    plt.ylabel("True positive rate")
    plt.savefig(os.path.join(PLOT_DIR, mname + ROC_PLOT_FNAME))
```

```
# Function to dump the model object for later use.
def dump_model(mname):
    with open(os.path.join(MODEL_DIR, f"{mname}-{MODEL_FNAME}"), "wb") as f:
        dump(model_pipeline, f, protocol=5)


# Function for producing a tabulable structure
# from the return type of cross_validate.
def mean_std_stats(m_name, cv_res, *args):
    res = [m_name]
    for key in args:
        res.append(np.mean(cv_res[key]))
        res.append(np.std(cv_res[key]))
    return res


# Function to save a table in latex format.
def save_table(table, name, mname, headers=None, floatfmt=".3f"):
    with open(os.path.join(TABLE_DIR, f"{mname}-{name}-{TABLE_FNAME}"), "w") as f:
        f.write(tabulate(table, tablefmt="latex", headers=headers,
floatfmt=floatfmt))


# Logging function
def log(msg, f, *args, **kwargs):
    print(msg, end=" ", flush=True)
    res = f(*args, **kwargs)
    print("DONE!")
    return res


# Read raw data
data = pd.read_csv(DATA_PATH)

# Assign the feature and label vectors
X = data["text"]
y = data["sentiment"]

# Split the data to training and testing subsets.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
dims = [
    ["Total dim.", X.shape],
    ["Train dim.", X_train.shape],
]

# Initialize the TF-IDF vectorizer for word embedding.
vectorizer = TfidfVectorizer(
    stop_words="english", max_features=5000, token_pattern="\\w+|[^\\w\\s]"
)

# Model pipeline definitions.
models = [
    ("LogReg", LogisticRegression(max_iter=1000)),
    ("RandForest", RandomForestClassifier(max_depth=None, criterion="log_loss")),
]
```

```
model_pipelines = [
    (name, Pipeline([("transformer", vectorizer), ("classifier", model)]))
    for name, model in models
]


cv_stats = []
metrics = []

print(tabulate(dims))

for model_name, model_pipeline in model_pipelines:

    # Fitting the model

    print("\n" + model_name.upper() + "\n")
    log("Fitting...", model_pipeline.fit, X_train, y_train)

    # K-Fold cross validation

    cv_res = log(
        "Cross validating...",
        cross_validate,
        estimator=model_pipeline,
        X=X,
        y=y,
        cv=5,
        n_jobs=5,
        scoring=["accuracy", "precision"],
    )

    # Compute stats from the K-fold CV

    stats = mean_std_stats(
        model_name,
        cv_res,
        "test_accuracy",
        "test_precision",
        "fit_time",
    )

    # Use the fitted model and compute some metrics

    y_pred = model_pipeline.predict(X_test)
    y_train_pred = model_pipeline.predict(X_train)
    y_score = model_pipeline.predict_proba(X_test)[:, 1]
    metric = [
        ["Training error", 1 - accuracy_score(y_train, y_train_pred)],
        ["Validation error", 1 - stats[1]],
        ["Test error", 1 - accuracy_score(y_test, y_pred)],
        ["Test precision", precision_score(y_test, y_pred)],
        ["Test ROC AUC", roc_auc_score(y_test, y_pred)],
    ]
    print(tabulate(metric))

    # Append acquired metrics and stats to table.
```

```
        cv_stats.append(stats)
        metrics.append([model_name] + [score for _, score in metric])

        # Save plots and the model for interaction

        log(
            f"Saving confusion matrix plot...",
            cm_plot,
            y_test,
            y_pred,
            model_name,
        )

        log(
            f"Saving ROC plot...",
            roc_plot,
            y_test,
            y_score,
            model_name,
        )

        log(f"Saving model...", dump_model, model_name)

    # Table headers...

    cv_headers = [
        "Model",
        "Mean acc.",
        "StD acc.",
        "Mean prec.",
        "StD prec.",
        "Mean train t (s)",
        "StD train t (s)",
    ]

    metrics_headers = [
        "Model",
        "Train error",
        "Valid. error",
        "Test error",
        "Test prec.",
        "Test ROC AUC",
    ]

    # Save the tables ready in LaTeX format.

    save_table(cv_stats, "cv", "total", headers=cv_headers)
    save_table(metrics, "metrics", "total", headers=metrics_headers)
```